

网站维护手册

赵洋摄影作品集 · yangzhaophoto.com

写给未来的你自己。这份手册假设你已经忘了所有细节，从零开始也能把网站维护起来。

序：这份手册管什么

网站已经建好并上线了。剩下的日子里你要做的事，无非三类：

1. 加东西——新拍的照片进 Gallery、Archive 或 Film
2. 改东西——填文案、调文字、换分享图
3. 发布——把改动同步到服务器，让访客看到

这份手册按三部分展开：**第一部分**讲 SSH 和远程同步（怎么把东西送上服务器）；**第二部分**是维护和更新的完整流程（日常怎么干活）；**第三部分**讲手动操作和背后的原理（万一没有 AI 助手，你自己也能做）。

另有一份《画册维护手册》在同一个文件夹里，专讲画册页（zine.html）的版式、插页、OG 卡片。那份更细，这份更全。两份不重复。

你的网站长什么样（一分钟回忆）

项目	内容
技术栈	纯 HTML + CSS + JavaScript，零构建（不需要 npm、不需要编译）
服务器	腾讯云轻量服务器，IP 124.221.92.171，宝塔面板管理
站点目录	/www/wwwroot/yangzhaophoto.com
代码仓库	GitHub yangcodingmaster/yang-photography（公开仓库）
唯一数据文件	data.js ——所有照片、系列、文案都在这里
唯一动态部分	api/message.php（留言表单），其余全是静态文件

核心心智：这个网站就是一堆文件。把文件放到服务器的正确位置，网站就更新了。没有数据库，没有后台，没有“发布按钮”。

第一部分：SSH 与远程同步

1.1 SSH 是什么

SSH 是一条加密的远程操作通道。

打个比方：服务器是一台放在深圳机房里的电脑，没有显示器、没有键盘。SSH 就是一根魔法电缆——你在自己的 Mac 上敲命令，命令通过这根电缆传到那台电脑上执行，结果再传回来显示在你的终端里。中间全程加密，别人截获也看不懂。

你用到 SSH 的场合有两个：

- 直接登上服务器操作（改配置、看日志、重启服务）
- `rsync` 同步文件（部署网站）——`rsync` 是骑在 SSH 这条通道上跑的

1.2 密钥登录：为什么不用密码

你的服务器已经关闭密码登录了（2026-07-26 做的加固），只认密钥。理解这件事的原理很重要，因为它是你能一条命令部署的前提。

类比：密码像一把钥匙的复印件——你每次开门都要把它掏出来，被人看到就完了。密钥登录像一把特制的锁和钥匙：

- 私钥（`~/.ssh/id_ed25519`）在你的 Mac 上，永远不离开你的电脑，相当于钥匙本体
- 公钥（`~/.ssh/id_ed25519.pub`）已经放在服务器上了，相当于门上的锁芯

连接时的过程是这样的：服务器出一道数学题，只有拿着私钥的人能算对。你算对了，门就开了——私钥本身从来没有在网络上传输过。所以哪怕整条线路被监听，别人也偷不走你的钥匙。

这也意味着一件事：你的 Mac 就是唯一的钥匙。换电脑要重新配密钥，Mac 丢了要立刻去服务器上撤销那把公钥。

你的密钥文件在哪

```
~/.ssh/id_ed25519      ← 私钥，绝对不能给任何人、不能进仓库、不能贴聊天
~/.ssh/id_ed25519.pub ← 公钥，可以随便给，它只是锁芯
```

万一要在新电脑上配置

```
ssh-keygen -t ed25519 -C "你的邮箱"
```

一路回车（密码短语可以留空）。然后把公钥送到服务器：

```
ssh-copy-id root@124.221.92.171
```

⚠ 但这一步需要密码登录，而你的服务器已经关闭密码登录了。所以新电脑配置密钥的正确路径是：先用腾讯云控制台的 VNC 登录（网页版的显示器+键盘，不走 SSH）进服务器，手动把新公钥粘贴进 `/root/.ssh/authorized_keys`。

1.3 常用 SSH 命令

登上服务器

```
ssh root@124.221.92.171
```

进去之后你就站在服务器的命令行里了。想回来，敲 `exit` 或按 `Ctrl+D`。

不登录，只执行一条命令

```
ssh root@124.221.92.171 "命令"
```

比如看网站目录里有多少文件：

```
ssh root@124.221.92.171 "ls /www/wwwroot/yangzhaophoto.com | wc -l"
```

这种用法很适合快速检查，不用真的登进去再退出来。

常用的服务器内操作

登进去之后：

想干什么	命令
去网站目录	<code>cd /www/wwwroot/yangzhaophoto.com</code>
看有哪些文件	<code>ls -lh</code>
看某个文件内容	<code>cat 文件名</code>
看磁盘还剩多少	<code>df -h</code>
看 Nginx 是否在跑	<code>systemctl status nginx</code>
重启 Nginx	<code>systemctl restart nginx</code>
看网站访问日志	<code>tail -50 /www/wwwlogs/yangzhaophoto.com.log</code>

1.4 rsync：只传变化的那部分

部署用的是 `rsync`，不是"上传整个文件夹"。

类比：假设你要把一本 600 页的书寄给朋友，而朋友手上已经有一本旧版。笨办法是重寄一整本（600MB，慢）。rsync 的办法是：先对比两本书，发现只有第 12 页和第 47 页改了，那就只寄这两页。

所以你改几行文案，部署就是几 KB、一两秒的事。只有第一次要传满 600MB。

1.5 `deploy.sh` 到底做了什么

`scripts/deploy.sh` 就是把 `rsync` 命令包装了一下，加了安全措施。它做四件事：

第一，同步文件。把你本地这个文件夹的内容推到服务器的站点目录。

第二，排除不该上公网的东西：

排除项	为什么
<code>.git</code>	版本历史，几百 MB，访客不需要
<code>.claude</code>	AI 助手的配置
<code>CLAUDE.md</code>	项目说明书，是给 AI 和你看的
<code>scripts</code>	维护脚本，不是网站的一部分
<code>api/config.php</code>	万一有人把密钥挪回这里的兜底保险
<code>api/README.md</code> 、 <code>api/config.sample.php</code>	不含密钥，但写着密钥存放路径，没必要公开

第三，`--delete`：让服务器成为本地的镜子。这是最需要理解的一条：本地没有的文件，服务器上会被删掉。

这是想要的行为——你删掉一张旧照片，服务器上也该消失，否则垃圾会越积越多。但它也是唯一的危险来源：如果脚本里的目标路径填错了，那个目录会被清空。所以换服务器或改配置之后，第一次一定要先 `--dry-run` 演习。

第四，归位文件属主。同步完自动在服务器上跑一次 `chown -R www:www`。原因是 Nginx 以 `www` 这个身份运行，文件理应属于它。（正规做法是给 `rsync` 加 `--chown` 参数，但 macOS 自带的 `rsync` 是 2006 年的老版本，没有这个参数，所以改成传完补一刀。）

1.6 密钥为什么不会被 `--delete` 删掉

留言表单的 Resend API key 是"只在服务器上存在、本地永远没有"的东西——正好是 `--delete` 要清理的对象。为此做了两层保险：

- **第一层（主要的）**：key 根本不在网站目录里，它在 `/www/site-secrets/config.php`。这个位置在站点目录外面，rsync 的镜子照不到。
- **第二层（兜底）**：脚本里那条 `api/config.php` 的排除规则。万一将来有人图省事把配置挪回 `api/` 目录，也不至于一部署就没了。

正常情况下第二层是多余的，留着是为了不正常的情况。

1.7 部署排障

症状	原因	怎么办
<code>Permission denied (publickey)</code>	密钥没配好，或换了电脑	检查 <code>~/.ssh/id_ed25519</code> 是否存在；用 VNC 登录服务器检查 <code>authorized_keys</code>
<code>Connection timed out</code>	服务器关机、IP 变了、或防火墙	腾讯云控制台看实例状态；确认 22 端口放行
<code>rsync: command not found</code>	服务器上没装 rsync	<code>ssh</code> 进去跑 <code>yum install rsync -y</code>
传完了但网页没变	浏览器缓存	Cmd + Shift + R 硬刷新
传完了但图片 404	忘了跑小图脚本	见第二部分 2.6

第二部分：网站维护与更新

2.0 完整工作流（先看全景）

不管你要改什么，流程都是这五步：

```
① 拉最新代码 → ② 改动 → ③ 提交存档 → ④ 部署上线 → ⑤ 验证
    git pull      改文件      git commit      deploy.sh      硬刷新看
```

为什么要走这么多步：git 负责"记住每个版本、随时能后悔"，deploy 负责"让访客看到"。两件事互相独立——push 到 GitHub 不等于上线，部署到服务器才是上线。

五步的具体命令

```
# ① 开工前先拉最新（多设备协作时防止冲突）
git pull

# ② .....改你的文件.....

# ③ 存档：先看改了些什么，再提交
git status
git add .
git commit -m "简短说明改了些什么"
git push

# ④ 部署：先演习，再真跑
bash scripts/deploy.sh --dry-run
bash scripts/deploy.sh

# ⑤ 打开 http://124.221.92.171 硬刷新验证
```

要不要开分支？小改动（填文案、加照片）直接在 main 上做。大改版（做好几天、中途不能让访客看到半成品）才开分支，做完合并回 main 再部署。判断标准只有一条：这个改动做到一半的样子，能不能被人看到？

2.1 往 Gallery 加一个新系列

Gallery 的每个"系列"= 一个照片文件夹 + `data.js` 里的一段描述。

最省事的办法

把照片文件夹丢进 `assets/images/gallery/`，然后跟 Claude 说：

「加一个系列，放在 XX 旁边，叫 XX」

会自动触发 `add-gallery-series` skill，它负责转码、压缩、命名、写 `data.js`、验证。

手动做的完整步骤

第 1 步：建文件夹。在 `assets/images/gallery/` 下新建，名字就是这个系列的 `id`。

⚠ `id` 必须用连字符，不能有空格（空格在网址里会变成 `%20`，又丑又容易出问题）。比如 `lake-district-collections`。

第 2 步：处理照片。相机直出的 HEIC 必须转成真 JPEG（HEIF 浏览器支持度低），并压缩到网页尺寸：

- 标准：长边 2560px、质量 90
- 命名：`01.jpeg`、`02.jpeg`按你想要的展示顺序

Mac 上可以用系统自带的 `sips` 命令批量转：

```
# 在照片文件夹里执行：HEIC 转 JPEG
for f in *.HEIC; do sips -s format jpeg "$f" --out "${f%.HEIC}.jpeg"; done

# 缩到长边 2560
sips -Z 2560 *.jpeg
```

⚠ 原始 HEIC 请在仓库外另存备份——入库的都是缩小过的网页版，将来印刷或二次修图要用原片。

第 3 步：写进 `data.js`。在 `allSeries` 数组里加一段：

```
{
  id:      'lake-district-collections',    // 必须和文件夹名一字不差
  titleZh: '在湖区',
  titleEn: 'Lake District Collections',
  year:    '2026',
  location: 'Windermere',
  descZh:  '中文简介',
  descEn:  '',
  cover:   'assets/images/gallery/lake-district-collections/07.jpeg',
},
```

⚠ **cover 是必填的**——所有系列都必须显式指定封面，不会自动取第一张。省略了 Gallery 页那一格就是空白。

第 4 步：在 `allPhotos` 里加照片列表：

```
'lake-district-collections': [
  {
    src:      'assets/images/gallery/lake-district-collections/01.jpeg',
    alt:      '',
    caption:  '',
    date:     '',
    location: '',
    desc:     '',
    meta:     '',
  },
  // .....每张一段
],
```

文字字段可以全部留空，以后想好了再填，空的自动不显示。**每个字段独占一行**——方便你以后手动编辑。

第 5 步：跑两个脚本（顺序不能反）：

```
python3 scripts/make-small-images.py    # 生成手机小图档
python3 scripts/make-zine-dims.py       # 更新画册的宽高表
```

第 6 步：本地看效果。用浏览器打开 `gallery.html`，**Cmd + Shift + R 硬刷新**（浏览器会缓存 `data.js`）。

分章系列（一个系列分几个章节）

结构不同的地方只有两处：

1. `allSeries` 那段要加一行 `type: 'sectioned'`

2. 照片写进 `allChapters` 而不是 `allPhotos` :

```
allChapters['photos-of-2025'] = [
  {
    id:      'the-summer',
    titleZh: '那个夏天',
    titleEn: 'The Summer',
    descZh:  '',
    descEn:  '',
    photos: [
      { src: '...', alt: '', caption: '', date: '', location: '', desc: '', meta: '' },
    ]
  },
];
```

照片文件夹也按章节分子文件夹。

2.2 往 Archive 加照片

Archive 是"散片档案", 按年份组织, **故意只放照片不放文案**。

第 1 步: 照片丢进 `assets/images/archive/YYYY/` (年份文件夹)。文件名随意——Archive 的顺序看数组, 不看文件名。

第 2 步: 在 `data.js` 的 `allArchive['YYYY']` 数组**最前面**加一行:

```
{ src: 'assets/images/archive/2026/whatever.jpg', alt: '' },
```

⚠ **放最前面 = 它成为这本书的封面和第一页。**

想配一句话页脚就加 `note` 字段:

```
{ src: '...', alt: '', note: '多佛白崖下的海湾' },
```

Archive 只认三个字段: `src`、`alt`、可选的 `note`。想给照片配标题/日期/地点/器材, 说明它该进 Gallery 而不是 Archive。

加新一年: 在 `allArchive` 里加 `'2026': []`, 书架上自动多一本书 (书脊粗细随照片数变化)。

第 3 步: 跑两个脚本 (同上), 刷新 `archive.html`。

2.3 往 Film 加一卷胶卷

第 1 步：建文件夹 `assets/images/film/年月-型号-格式/`，比如 `2607-portra400-135`。照片按拍摄顺序命名 `01.jpeg`、`02.jpeg`

第 2 步：在 `data.js` 的 `allFilms` 数组里加一段：

```
{
  id:      '2607-portra400-135',      // = 文件夹名
  stock:   'Kodak Portra 400',        // 型号全名，同型号必须一字不差（页面按它分排）
  label:   'PORTRA 400',              // 筒身印字，每个空格 = 换一行
  format:  '135',                    // '135' 或 '120'，决定筒的形状和片框有无齿孔
  camera:  'Canon EOS-1',
  lens:    '',
  date:    '2026.07',                // 精确到月
  title:   '',                       // 作品名（可空）
  desc:    '',                       // 这卷的一句解释（可空）
  photos: [
    { src: 'assets/images/film/2607-portra400-135/01.jpeg', alt: '' },
  ],
},
```

三条要注意的：

- 同型号自动归到同一排搁板，新型号自动多一排。同排内按数组顺序从左到右 = 时间顺序，所以早的放前面。
- ⚠ 新型号需要配色：如果 `stock` 是表里没有的型号，要去 `film.html` 的 `SHELL_STYLES` 补一行品牌配色，否则那卷会显示成灰调。
- `photos` 里只放 `src`、`alt`、可选 `note`。卷级的文字用 `title / desc`。

第 3 步：跑两个脚本，刷新 `film.html`。

2.4 改文案

所有文字都在 `data.js` 一个文件里。对照表：

想改什么	去哪改
系列标题、年份、地点	<code>allSeries</code> 里对应那段
系列简介	<code>allSeries</code> 的 <code>descZh</code> / <code>descEn</code>
某张照片的作品名	<code>allPhotos</code> 或 <code>allChapters</code> 里那张的 <code>caption</code>
照片的日期/地点/故事/器材	同上的 <code>date</code> / <code>location</code> / <code>desc</code> / <code>meta</code>
章节标题	<code>allChapters</code> 里的 <code>titleZh</code> / <code>titleEn</code>
Film 某卷的作品名/解释/镜头	<code>allFilms</code> 对应卷的 <code>title</code> / <code>desc</code> / <code>lens</code>
关于页自我介绍	<code>about.html</code> , 直接找中文段落改
浏览器标签页标题	各 <code>.html</code> 的 <code><title></code> 标签

改完直接部署，不需要跑任何脚本（没动图片）。

2.5 微信分享卡片（OG）：封面、标题、描述

把网址发到微信（聊天、朋友圈都一样），对方看到的不是一串光秃秃的网址，而是一张卡片：



这三样东西由每个 HTML 文件 `<head>` 里的三行标签决定：

```
<meta property="og:title"      content="作品集 · 赵洋摄影" />
<meta property="og:description" content="八个系列：从虚焦、湖区，到三年的照片记忆。" />
<meta property="og:image"     content="https://yangzhaophoto.com/assets/images/share/share-
```

这套标签是通用标准——微信、QQ、微博、飞书、Slack、iMessage 全都读它。改一次，所有平台同时生效。

每个页面各有一套（改哪个页面就打开哪个文件）

以下是当前实际的值，想改就直接在对应文件里搜 `og:title` 改 `content`：

文件	标题 (og:title)	描述 (og:description)
index.html	赵洋 · 摄影作品集	八个系列，五年散片，一间随时开放的展厅。
gallery.html	作品集 · 赵洋摄影	八个系列：从虚焦、湖区，到三年的照片记忆。
archive.html	散片档案 · 赵洋摄影	2021 到 2025，那些不成系列却舍不得丢的照片。
film.html	胶卷档案 · 赵洋摄影	七卷胶卷，一卷是一个时间胶囊。
about.html	关于 · 赵洋摄影	关于我，以及联系方式。（占位，待你改）
zine.html	作品集 · 赵洋摄影	八个系列：从虚焦、湖区，到三年的照片记忆。

封面图六个页面共用同一张：`assets/images/share/share-card.jpg`。

改标题

打开对应的 `.html`，找到 `og:title` 那一行，改引号里的字。

- 12 字以内，超了微信会截断
- 建议保留"· 赵洋摄影"这个后缀——朋友转发时别人一眼知道是谁的站

改描述

同一个文件里的 `og:description`。

- ⚠️ 手机上只显示约 15—20 个字，写长了后面全是白写
- 同一句话建议同步改同文件里的 `<meta name="description">`（那条是给搜索引擎看的，内容通常一致）
- 写法上：一句话说清"这页有什么"，不要写成口号

举例，改 About 页的描述：

```
<!-- 改前（占位） -->
<meta name="description" content="关于我，以及联系方式。" />
<meta property="og:description" content="关于我，以及联系方式。" />

<!-- 改后 -->
<meta name="description" content="在北京和伦敦之间成长，用照片缅怀当下。" />
<meta property="og:description" content="在北京和伦敦之间成长，用照片缅怀当下。" />
```

换封面照片

封面不是直接放照片，而是用一个"模具"生成的——暖白纸面 + 居中照片（3px 黑描边）+ Cormorant 落款 "Yang"，和网站同一套字体和颜色。

1. 打开 `scripts/make-share-card.sh`，改里面的 `PHOTO` 那一行，指向你想用的照片
2. 跑一次：

```
bash scripts/make-share-card.sh
```

(favicon 和 apple-touch-icon 会一并重新生成)

1. 提交 + 部署

选图标准：高对比、构图简单。缩略图在微信里只有百来像素，细节丰富的照片缩到那个尺寸会糊成一团。当前用的是黑白柱廊逆光那张。

为什么卡片里的照片比网站上显得大：这是有意的。网站那种奢侈留白在百来像素下会显得空、不抓眼。

模具支持三种版式（在脚本里切换）：`center`（当前用）、`polaroid`（照片偏上、下方厚边）、`icon`（做图标用）。

微信的几条特殊规矩

微信的卡片规则和 Facebook/Twitter 不一样，设计要迁就它：

规矩	说明
缩略图被裁成正方形	所以封面图做成 1:1（当前 1200×1200）
图必须公网可访问	且备案通过后要求 https ——现在只有 http，微信可能抓不到图
描述只显示一行	约 15–20 字
抓取结果会缓存	改了不一定立刻变，见下面

改完不生效？

微信服务器会缓存抓取结果。强制重新抓取的办法：**在发出去的网址后面加一个参数。**

```
https://yangzhaophoto.com/gallery.html?v=2
```

下次再改就用 `?v=3`。参数不影响网页本身，只是骗微信“这是个新网址”。

两条硬约束（改之前必读）

第一，`og:image` 和 `og:url` 必须写完整网址（带 `https://` 和域名）。

这是全站**唯一**允许出现**绝对路径**的地方。原因：微信服务器是拿着这个网址去下载缩略图的，写相对路径它找不到。

```
换域名时不用手改——跑 python3 scripts/set-domain.py 新域名.com，它会把全站 12 处一起替换。
```

第二，微信抓取时不执行 JavaScript。

所以卡片内容必须写死在 HTML 里，不能靠脚本生成。这条约束带来一个当前做不到的事：

为什么每个系列不能有自己的分享卡片

你可能想要：分享"再见，UCL"这个系列时，卡片显示这个系列自己的封面和标题。

现在做不到，原因是架构性的：`zine.html` 一个文件服务全部八个系列，靠网址里的 `?id=` 区分。但查询参数不改变服务器返回的文件——微信抓到的永远是同一个 `zine.html`，读到的永远是同一套标签。加多少 `og` 标签都没用。

Archive 和 Film 更麻烦：点开某一年/某一卷现在也是 `?year=` `?film=` 参数形式。

能做到需要改成静态多页：写个脚本从 `data.js` 批量生成 `series/goodbye-ucl.html` 这样的文件，每个里面写死自己的 `og` 标签。生成物直接进仓库，网站仍是纯静态、仍不需要 `npm`。专属分享图也不用另做模具，`scripts/share-card.html` 换个 `photo` 参数就能批量出。

这件事已列入 `CLAUDE.md` 的"未来可能的方向"，你说一声就能做。

怎么验证卡片效果

改完部署后，把网址发到微信的"文件传输助手"（发给自己），看卡片长什么样。这是最直接的验证方式，不用麻烦别人。

⚠ 备案通过、配好 HTTPS 之前，微信可能抓不到图（它要求 `https`）。所以现在看到卡片没图是正常的，不是你改错了。

2.6 两个必须知道的脚本

`make-small-images.py` ——手机小图档

网站的图片是两档制：

- 原图档 `assets/images/`：长边 2560、质量 90，给高分屏桌面
- 小图档 `assets/images-sm/`：长边 1280、质量 85，给手机

为什么要这样：手机屏幕最多用到 1300px 宽，而服务器出口只有约 400KB/s。小图把翻页等待从 3 秒降到 1 秒内。

```
python3 scripts/make-small-images.py          # 增量（加了照片跑这个）
python3 scripts/make-small-images.py --force   # 全部重做（改了参数才用）
```

⚠ 加了照片一定要跑。页面是按路径规则自动推导小图地址的（`assets/images/` → `assets/images-sm/`），缺一张小图就是一个 404 破图。脚本末尾的数量自检必须显示"一致"。

⚠️ **glitch 作品不会被重编码**——脚本自动识别（靠码流检测），原字节拷贝。因为不同解码器渲染损坏码流的结果不同，重编码等于替你选定了一种渲染。

make-zine-dims.py ——画册宽高表

画册页给每张图预先写好宽高，图片没加载完时页面也不跳版。

```
python3 scripts/make-zine-dims.py
```

必须在小图脚本之后跑（它从小图里读尺寸）。生成的 `zine-dims.js` 是机器生成的，别手改。

记住这个顺序

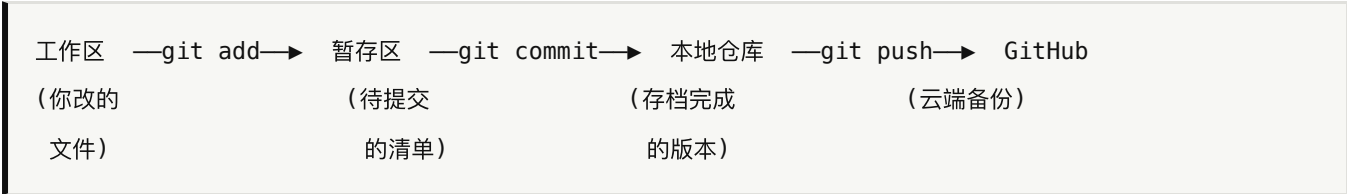
```
加了照片 → make-small-images.py → make-zine-dims.py → 部署
```

第三部分：手动操作与原理

这部分回答一个问题：如果哪天没有 AI 助手，你自己怎么办？

3.1 最少必要知识：Git 的心智模型

Git 你只需要理解四个地方和文件在它们之间的流动：



类比：写论文时，工作区是"正在编辑的文档"，暂存区是"选中要保存的段落"，commit 是"另存为一个带日期的版本"，push 是"上传到云盘"。

你只需要这几条命令

命令	干什么	什么时候用
git status	看现在改了哪些文件	每次动手前后都看一眼
git diff	看具体改了哪几行	提交前确认
git add .	把所有改动放进暂存区	提交前
git commit -m "说明"	存一个版本	每完成一件事
git push	传到 GitHub	存完档
git pull	从 GitHub 拉最新	每次开工前
git log --oneline	看历史版本列表	想找某次改动

后悔药（按后悔程度排序）

```
# 改坏了某个文件，还没 commit：把它恢复到上次提交的样子
git checkout -- 文件名

# 所有改动都不要了，回到上次提交的状态
git reset --hard

# 已经 commit 了但想改提交说明
git commit --amend -m "新说明"

# 已经 commit 了想撤销这次提交（保留文件改动）
git reset --soft HEAD~1

# 想回到某个历史版本看看（先 git log 找到那串编号）
git checkout 版本编号
git checkout main          # 看完回来
```

最重要的习惯：每达到一个可用状态就 commit，大改动前先 commit。这样"回滚"永远只有一条命令的距离。

3.2 分支的本质

分支不是复制文件夹，它只是一个指向某次提交的书签。

```
main: ●—●—●—●—● ← 正式版，部署用这条
      \      /
新分支: ●—●—● ← 试验区，改砸了直接扔掉
```

```
git checkout -b 新分支名    # 建分支并切过去
git checkout main          # 切回主分支
git merge 新分支名         # 把分支的改动合并进当前分支
git branch -d 新分支名     # 删掉已合并的分支
```

这个仓库的特殊性：push 到 main 会触发 GitHub Pages 自动更新（那是海外镜像）。而腾讯云只在你手动跑 deploy.sh 时才更新。两者互相独立。

3.3 如果 `deploy.sh` 挂了：手动同步

脚本只是 `rsync` 的包装。手动跑等价命令：

```
rsync -avz --progress --delete \  
  --exclude '.git' \  
  --exclude '.claude' \  
  --exclude 'CLAUDE.md' \  
  --exclude 'scripts' \  
  --exclude '.gitignore' \  
  --exclude '.DS_Store' \  
  --exclude 'api/config.php' \  
  --exclude 'api/README.md' \  
  --exclude 'api/config.sample.php' \  
  ./ root@124.221.92.171:/www/wwwroot/yangzhaophoto.com/
```

然后归位属主：

```
ssh root@124.221.92.171 "chown -R www:www /www/wwwroot/yangzhaophoto.com"
```

参数含义：`-a` 保留文件属性、`-v` 显示过程、`-z` 传输时压缩、`--progress` 显示进度、`--delete` 让服务器成为本地的镜子。

⚠ 最后那个斜杠很重要。`./` 表示"当前目录的内容"；如果写成 `.`（没有斜杠），`rsync` 会把整个文件夹作为一个子目录塞进去，变成 `/www/wwwroot/yangzhaophoto.com/我的展览/`。

只传单个文件（应急改一行）

```
scp 本地文件 root@124.221.92.171:/www/wwwroot/yangzhaophoto.com/文件名
```

比如紧急改了 `data.js`：

```
scp data.js root@124.221.92.171:/www/wwwroot/yangzhaophoto.com/data.js
```

⚠ 这样改完要记得在本地也提交 `git`，否则下次 `rsync` 会把服务器上的版本覆盖回去。

3.4 如果连宝塔面板也进不去

宝塔只是个网页版的操作界面，它能做的事 `SSH` 都能做：

宝塔里的操作	命令行等价
网站 → 设置 → 重载	<code>systemctl reload nginx</code>
文件管理器浏览	<code>ls -lh /www/wwwroot/yangzhaophoto.com</code>
看错误日志	<code>tail -50 /www/wwwlogs/yangzhaophoto.com.error.log</code>
重启 PHP	<code>systemctl restart php-fpm-82</code>
看磁盘占用	<code>df -h</code>

如果连 SSH 也进不去

腾讯云控制台 → 轻量应用服务器 → 你的实例 → 登录 (VNC)。

这是网页版的显示器和键盘，不走 SSH，所以哪怕 SSH 配置被改坏了也能进。SSH 加固的备份在服务器的 `/root/ssh-backup-20260726-050007`，想恢复密码登录：

```
rm /etc/ssh/sshd_config.d/00-hardening.conf
systemctl reload sshd
```

3.5 网站打不开了怎么排查

按这个顺序查，从外到内：

第 1 步：是不是只有你打不开？

```
curl -I http://124.221.92.171
```

返回 `HTTP/1.1 200 OK` 说明服务器是好的，问题在你这边（缓存、网络）。

第 2 步：服务器活着吗？

```
ssh root@124.221.92.171 "uptime"
```

连不上 → 去腾讯云控制台看实例是否关机/欠费。

第 3 步：Nginx 在跑吗？

```
ssh root@124.221.92.171 "systemctl status nginx"
```

没跑 → `systemctl start nginx`。

第 4 步：文件还在吗？

```
ssh root@124.221.92.171 "ls /www/wwwroot/yangzhaophoto.com/index.html"
```

不在 → 说明某次 rsync 出了问题，重新跑一次 `deploy.sh`。

第 5 步：看错误日志

```
ssh root@124.221.92.171 "tail -50 /www/wwwlogs/yangzhaophoto.com.error.log"
```

3.6 为什么是这套架构（原理小结）

理解这几条，你就能自己判断大部分问题：

为什么零构建。网站是纯 HTML/CSS/JS，浏览器直接就能跑。不需要 npm、不需要编译。好处是：五年后你再打开这个文件夹，它照样能用——不会因为某个工具链过时而全盘崩塌。代价是不能用现代框架，但对一个作品集网站来说这个交易很划算。

为什么所有内容在一个 `data.js`。因为你要填的是文案，不是代码。一个文件、一处修改、四个页面同时生效。

为什么图片要两档。服务器出口带宽约 400KB/s，一张 2560px 的照片 1.16MB，手机上要等 3 秒。小图档 1280px 只要 0.3MB，等待降到 1 秒内。这不是优化，是可用性。

为什么境外资源全部本地化。`fonts.googleapis.com` 在国内确定被墙，`cdn.tailwindcss.com` 时通时断。如果在线引用，国内访客看到的会是没有样式的裸文字。所以 Tailwind 和字体都存了本地副本在 `assets/vendor/`。别改回在线引用。

为什么密钥在网站目录外面。因为 `rsync --delete` 会让服务器成为本地的镜子，任何“只在服务器上存在”的文件都会被删掉。把 key 放在 `/www/site-secrets/` 是唯一安全的位置。

为什么 GitHub 和服务器的两回事。GitHub 是代码仓库（版本历史 + 备份），服务器是网站运行的地方。`git push` 只更新前者。只有 `deploy.sh` 才让访客看到新内容。

附录：速查表

日常更新（改文案）

```
git pull                                # 拉最新
# .....改 data.js 或 html.....
git add . && git commit -m "改了什么" && git push
bash scripts/deploy.sh --dry-run        # 演习
bash scripts/deploy.sh                  # 真跑
# 打开 http://124.221.92.171 硬刷新验证
```

加了照片之后

```
python3 scripts/make-small-images.py    # 生成小图（必须）
python3 scripts/make-zine-dims.py        # 更新宽高表（必须）
git add . && git commit -m "加了 XX 系列" && git push
bash scripts/deploy.sh
```

改微信分享卡片

想改	去哪
标题	对应 <code>.html</code> 的 <code>og:title</code> （12 字以内）
描述	同文件的 <code>og:description</code> + <code><meta name="description"></code> （15—20 字）
封面照片	改 <code>scripts/make-share-card.sh</code> 的 <code>PHOTO</code> 行 → 跑 <code>bash scripts/make-share-card.sh</code>
改完没变	网址后加 <code>?v=2</code> 强制微信重抓

服务器速查

```
ssh root@124.221.92.171                # 登录
ssh root@124.221.92.171 "systemctl status nginx" # 查 Nginx
ssh root@124.221.92.171 "df -h"         # 查磁盘
curl -I http://124.221.92.171           # 查网站是否响应
```

关键路径

东西	位置
本地项目	/Users/yang/Downloads/我的展览
服务器站点	/www/wwwroot/yangzhaophoto.com
服务器密钥	/www/site-secrets/config.php （不在仓库里）
SSH 私钥	~/.ssh/id_ed25519 （永不外传）
SSH 加固备份	/root/ssh-backup-20260726-050007
网站日志	/www/wwwlogs/yangzhaophoto.com.log

三条铁律

- 部署前先 `--dry-run`，看清单再真跑。
- 加了照片必须跑两个脚本，否则手机端全是破图。
- 密钥永远不进仓库、不进聊天。仓库是公开的。

本手册基于仓库中真实的脚本与配置编写。项目结构和更细的设计约定见 `CLAUDE.md`；画册页的版式教程见 `画册维护手册.md`。